CS-137 Assignment 3: Multi-Modal Deep Learning for Energy Demand Forecasting

Goal

The goal of this assignment is to gain hands-on experience fusing high-dimensional spatial data with tabular sequential data to solve a critical real-world application: day-ahead electricity demand forecasting. You will build a state-of-the-art hybrid CNN-Transformer. By downsampling weather maps into spatial "patches" and combining them with grid data into a unified sequence, your model will learn to attend to both geographic weather patterns and human scheduling to predict future energy loads.

Project Description

In this project, you will build a multi-modal neural network to predict the energy demand for the next 24 hours for **all** load zones across the ISO New England grid.

In real-world grid operations, demand predictions rely heavily on weather forecasts. We will emulate this by assuming the weather can be accurately forecasted, allowing us to use the *true* future weather data as inputs. Given a historical window ending at time t, your task is to use historical weather and demand, alongside future "forecasted" weather and calendar features, to predict the demand sequence for $t+1$ through $t+24$.

Problem Formulation & Inputs

Let S be the length of your historical look-back window, and let Z denote the number of ISO New England load zones. Your model will receive the following inputs:

1. **Historical Weather & Demand:**
   - Weather Snapshots: $X_{t-S}, \ldots, X_t$ (Tensors of shape $450 \times 449 \times c$)
   - Actual Energy Demand: $Y_{t-S}, \ldots, Y_t$ (Length-Z vectors containing per-zone MWh values)
   - Calendar Features: $C_{t-S}, \ldots, C_t$ (e.g., hour, day, holiday status)
2. **Future (Forecasted) Weather & Metadata:**
   - Weather Snapshots: $X_{t+1}, \ldots, X_{t+24}$
   - Calendar Features: $C_{t+1}, \ldots, C_{t+24}$

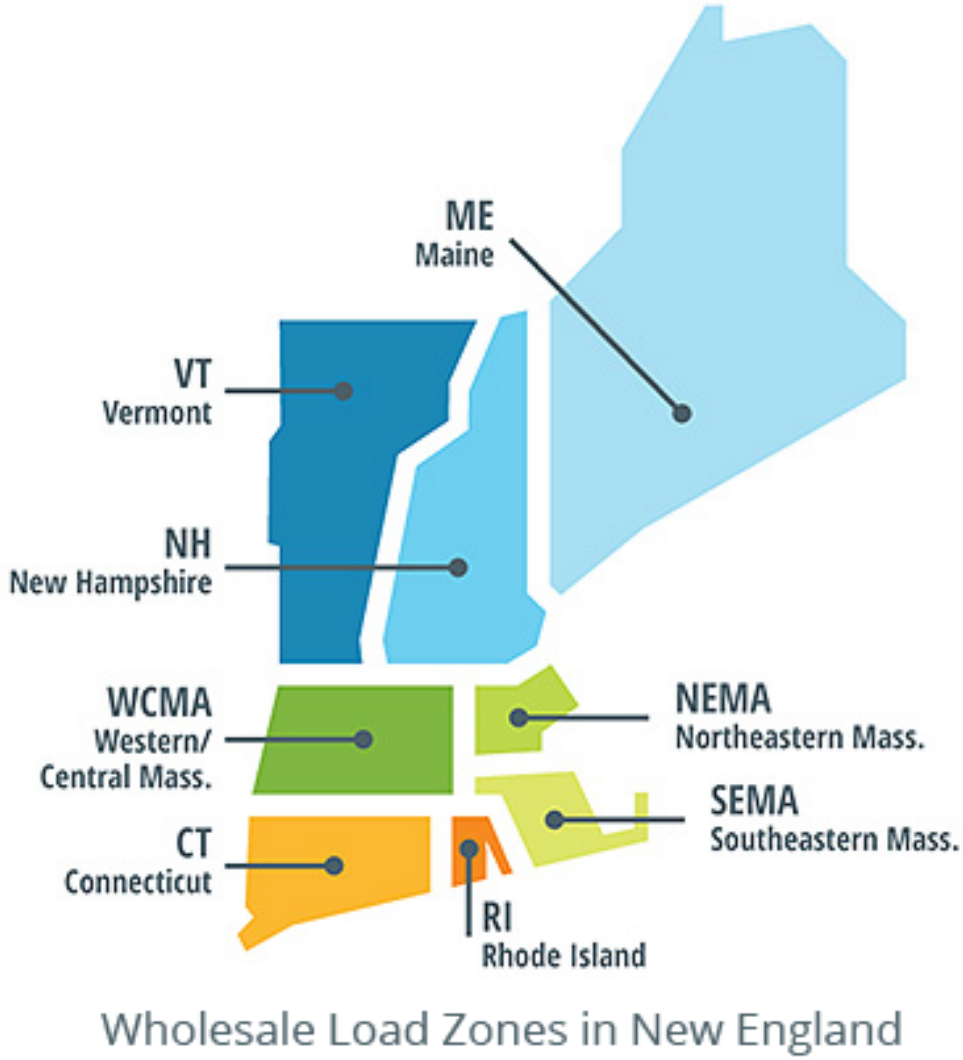

Figure 1: Spatial area of the study overlaying the ISO New England Load Zones.

Prediction Target

The model should predict a sequence of 24 vectors representing the energy demand for the next 24 hours across all load zones:

$$\hat{Y}_{t+1}, \hat{Y}_{t+2}, \ldots, \hat{Y}_{t+24}, \qquad \text{where each } \hat{Y}_{t+i} \text{ is a length-}Z \text{ vector}$$

Evaluation

We have set up an evaluation protocol in the `evaluation/` folder. You will need to provide a `model.py` file containing a `get_model` function.

While weather models often use RMSE, the energy industry standard for demand forecasting is **Mean Absolute Percentage Error (MAPE)**. We will evaluate all models using the average MAPE across both the 24-hour forecast horizon and all load zones on a held-out test set. *(Note: You may use MSE or MAE as your training loss function, as MAPE can sometimes be unstable*

for gradient descent).

$$\text{MAPE} = \frac{1}{24Z} \sum_{i=1}^{24} \sum_{z=1}^{Z} \left| \frac{Y_{t+i}^{(z)} - \hat{Y}_{t+i}^{(z)}}{Y_{t+i}^{(z)}} \right| \times 100\%$$

Data

All data are located at `/cluster/tufts/c26sp1cs0137/data/assignment3_data`.

The dataset covers hourly data from 2019, to 2023. All timestamps have been aligned in UTC to match the meteorological inputs and avoid Daylight Saving Time inconsistencies. The data files provided include:

- **Weather Tensors:** The sequence of $(450, 449, c)$ weather tensors.
- **Energy Demand CSVs:** The hourly energy demand for all ISO New England load zones. *Tip: You are strongly encouraged to normalize these values (e.g., z-score) before feeding them into your network.*

We will evaluate your models on held-out test data from 2024.

Grading (100 Points)

Part 1: Baseline CNN-Transformer Patch Architecture (40 points)

Implement a baseline hybrid architecture using spatial patch tokens.

- **Spatial Tokens (All timesteps):** For every hour (both historical and future), pass the weather map X_i through a CNN to downsample it to a smaller grid (e.g., 10×10). Flatten this grid into spatial tokens. *Warning: Your sequence length will be $(S+24) \times (P+1)$, where P is the number of spatial tokens. Transformer memory scales quadratically. If your CNN grid P is too large (e.g., 50×50), your sequence will be massive and you will run out of GPU memory. Start with a small grid frist.*
- **Historical Tabular Tokens ($t-S$ to t):** Concatenate historical demand (Y_i) and calendar features (C_i), passing them through a linear layer.
- **Future Tabular Tokens ($t+1$ to $t+24$):** Pass only the future calendar features (C_i) through a linear layer (padding the missing multi-zone Y_i input with zeros or a learned masking vector).
- **Transformer Sequence Assembly:** Project all spatial and tabular tokens to the same embedding dimension, D. Add learnable spatial positional embeddings to both the historical and future spatial tokens so that nearby or corresponding grid locations can receive similar encodings. For each hour, group your P spatial tokens and your 1 tabular token. Concatenate these groups chronologically across all $S + 24$ timesteps to form a single, long sequence of length $(S + 24) \times (P + 1)$. Then add a per-timestep positional encoding to every token in that hour's group so that all data from the same timestep shares the same temporal encoding before passing the full sequence through a Transformer Encoder.
- **Prediction:** Slice out the final Transformer states corresponding *only* to the 24 future hours and pass them through an MLP to predict the multi-zone outputs $\hat{Y}_{t+1:t+24}$.

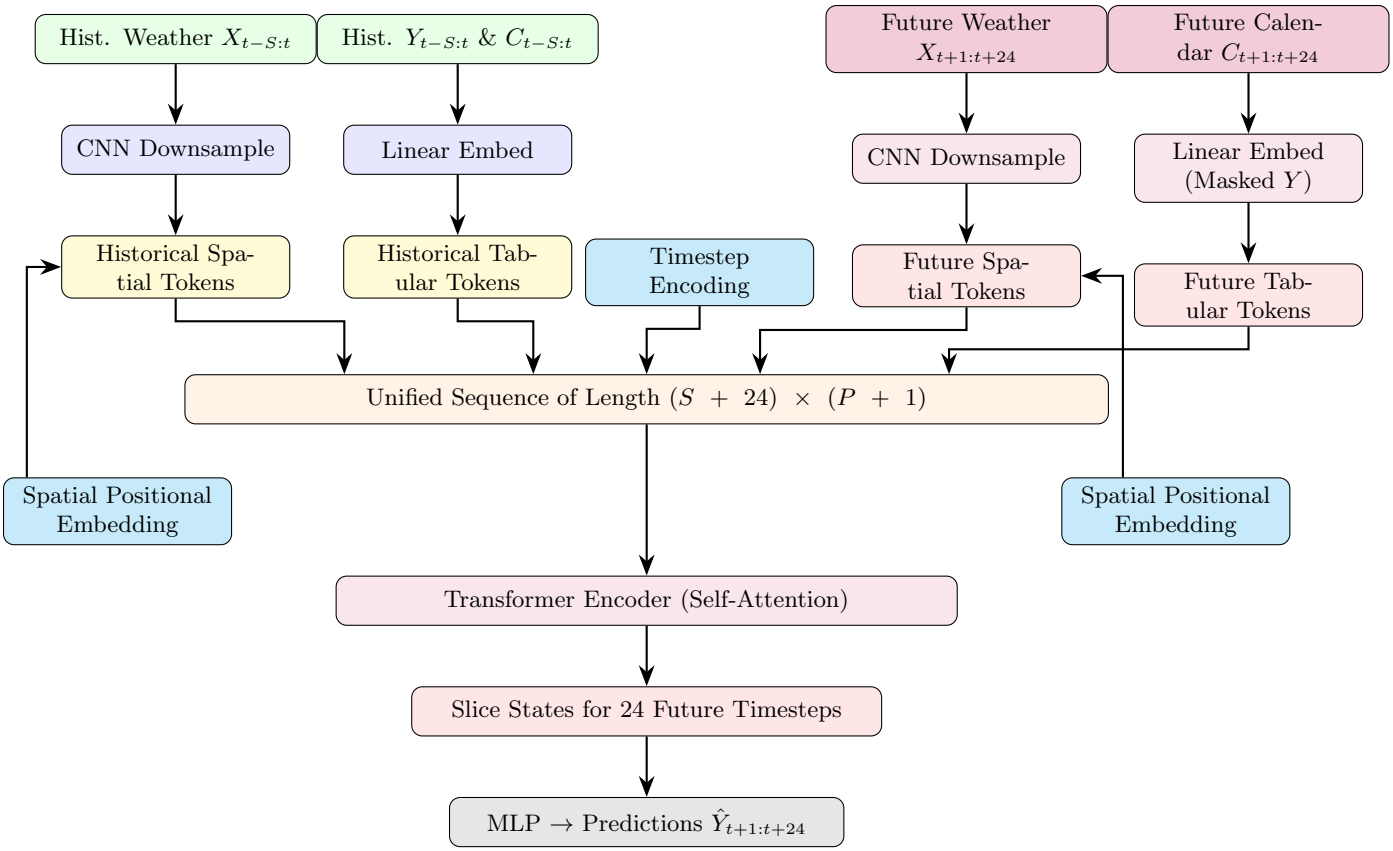

Figure 2: Sequence-to-Sequence Hybrid Architecture for Day-Ahead Forecasting

Part 2: Architecture Search & Beating the Baseline (30 points)

Attempt to Design a new architecture that outperforms your Part 1 baseline. Since your baseline is already highly capable, you will need to think critically about spatial-temporal bottlenecks. Examples include:

- **Alternative Architectures:** Try different architectures such as recurrent models.
- **Hierarchical Attention:** Build a spatial Transformer to summarize the weather per hour first, followed by a temporal Transformer for the $S + 24$ sequence.
- **Encoder-Decoder:** Separate the historical sequence ($t - S$ to t) into a Transformer Encoder, and feed the future weather/calendar ($t + 1$ to $t + 24$) into a Transformer Decoder (Cross-Attention).
- **Pre-training/Feature Engineering:** Engineer localized weather features or use pretrained models for transfer learning.

Deliverable: A comparative analysis against the baseline and a discussion on why the new architecture performed differently.

Part 3: Model Diagnosis or Independent Study (30 points)

Choose **one** of the following tracks for your final investigation:

- **Track A - Geographic Attention Maps:** Because your model uses spatial tokens, extract the attention weights between the Future Tokens (the prediction hours) and the Spatial Tokens. Reshape these weights back into a 2D map. What geographic regions drive demand across different ISO New England load zones? Does the model "track" incoming weather systems across the map, and do different zones attend to different regions?
- **Track B - Independent Study:** Formulate and test an independent hypothesis. Examples include evaluating performance exclusively during extreme weather events, more extensive architecture design/transfer learning etc.

Working in Groups

Students must work in groups. The maximum group size is 3 students. Each group submits a single report and codebase. All group members are expected to contribute and should briefly describe their individual contributions in the submission.

Deliverables

- **Code:** A well-organized repository or zip file containing all training scripts.
- **Report:** A written report (PDF) covering Parts 1, 2, and 3.
- **Presentation:** A short presentation (up to 7 pages).
- **Model checkpoint:** Your best trained model weights.

Milestones

All milestones are due **end of day (11:59pm EST)**.

Date	Milestone
April 15	Part 1
April 22	Part 2
May 1	All code & report, and experiments
May 4	Presentation Submission

Due time

Submit all written/code deliverables to GradeScope by **May 1, 11:59pm EST**. The presentation is due by **May 4, 11:59pm EST**. Presentations will be held on **May 5, 12:00pm-2:00pm EST**.